```
  }

  const currentStatus = (): string => {
    if (seatMatch(context.state.otherTickets, true)) {
      return "purchased"
    }
    if (seatMatch(context.state.myTickets, true)) {
      return "held"
    }
    if (
      seatMatch(context.state.otherTickets) ||
      seatMatch(context.state.myTickets) ||
      seatNumber + context.state.ticketsToBuyCount - 1 >
        context.state.seatsPerRow
    ) {
      return "invalid"
    }
    return "unsold"
  }

  const onSeatChange = (): void => {
    const status = currentStatus()
    if (status === "invalid" || status === "purchased") {
      return
    }
    const actionType = status === "unsold" ? "holdTicket" : "unholdTicket"
    context.dispatch({ type: actionType, seatNumber, rowNumber })
  }

  return (
    <td>
      <ButtonSquare status={currentStatus()} onClick={onSeatChange}>
        {seatNumber}
      </ButtonSquare>
    </td>
  )
}

export default Seat
```

A lot of this is similar to the previous version of `Seat`. The new bits are the `currentStatus` function and its attendant `seatMatch` function. These two functions are the new home of the business logic for determining if the seat is available, using